

Single-layer NN/Shallow NN

Regression and Classification

Fundamentals of Deep Learning · Class 5 · Regression

3 August

Outline

Regression with Fixed Features

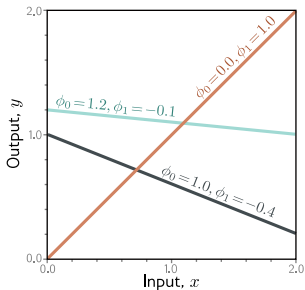
An Example Shallow Network

Generalising the Network

Training

Regression with Fixed Features

Linear regression, two views



Prince (2023), Fig. 2.2.

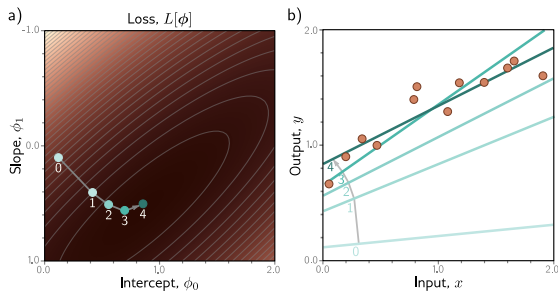
$$y = f[x, \phi] = \phi_0 + \phi_1 x$$

- Two parameters: intercept ϕ_0 , slope ϕ_1
- Fit by minimising the **least-squares loss**
- A **family** of lines; parameters select one

ML model = family of functions

NN parameters select one member — this framing carries through the whole course

Training: minimise the loss



Prince (2023), Fig. 2.4.

$$L[\phi] = \sum_i (f[x_i, \phi] - y_i)^2$$

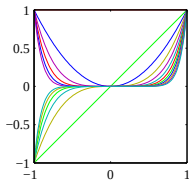
- Loss surface over (ϕ_0, ϕ_1) ; each point is one candidate line
- Walk downhill: **gradient descent**
- For a *linear* model this is convex — one minimum, closed form exists

Beyond lines: fixed basis functions

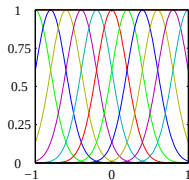
Standard extension (Bishop & Bishop, 2024, §4.1):

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}) \quad (\text{B\&B 4.3})$$

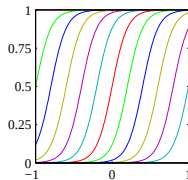
The ϕ_j are chosen **before** seeing data; only $\mathbf{w}$ is learned.



polynomials



Gaussians



sigmoids

Bishop & Bishop (2024), Fig. 4.2.

Least squares in closed form

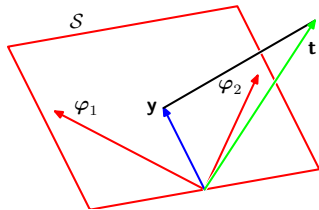
Stack the basis functions into the **design matrix**

$\Phi_{nj} = \phi_j(\mathbf{x}_n)$ (Bishop & Bishop, 2024, Eq. 4.15).

Setting $\nabla_{\mathbf{w}}$ of the error to zero:

$$\mathbf{w}_{\text{ML}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t} \quad (\text{B\&B 4.14})$$

- The **normal equations** — possible because the model is **linear in $\mathbf{w}$**



Bishop & Bishop (2024), Fig. 4.3: $\Phi \mathbf{w}_{\text{ML}}$ is the orthogonal projection of $\mathbf{t}$.

ML closed form exists (Eq. 4.14)

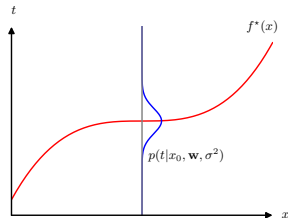
NN parameters inside $\mathbf{a}[\cdot]$ will remove the closed form

Why least squares? The probabilistic view

Assume Gaussian noise around the model output
(Bishop & Bishop, 2024, §4.1.1–4.1.3):

$$p(t \mid \mathbf{x}, \mathbf{w}, \sigma^2) = \mathcal{N}(t \mid y(\mathbf{x}, \mathbf{w}), \sigma^2) \quad (\text{B\&B 4.8})$$

- Max. log likelihood (Eq. 4.10) $\equiv$ min.
sum-of-squares (Eq. 4.11)
- Least squares = maximum likelihood,
Gaussian noise

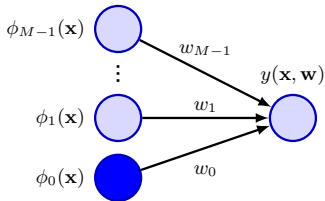


Bishop & Bishop (2024), Fig. 4.5: the optimal prediction is the mean of $p(t \mid x)$.

ML loss $\leftarrow$ noise model: Gaussian $\rightarrow$
squared error

NN Class 6 reuses this logic: Bernoulli $\rightarrow$
cross-entropy

Linear regression is itself a network



Bishop & Bishop (2024), Fig. 4.1.

- Nodes: basis functions $\phi_j(\mathbf{x})$; links: weights w_j
- Already a one-layer network diagram — but the ϕ_j carry no parameters

The limitation

For images, audio, text — nobody knows how to write good ϕ_j by hand.

ML ϕ_j fixed by design

NN next: make the basis functions *adaptive*

Why shallow networks

A linear model in fixed features is limited. We want a model that is (Prince, 2023, Ch. 3 opening):

- flexible enough to describe **arbitrarily complex** input–output mappings,
- able to take **as many inputs** as we want,
- able to produce **as many outputs** as we want.

Shallow neural networks do all three — by putting parameters *inside* the basis functions and learning them from data.

An Example Shallow Network

The model

Scalar input x , scalar output y , ten parameters

$\phi = \{\phi_0, \phi_1, \phi_2, \phi_3, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}, \theta_{30}, \theta_{31}\}$:

$$y = f[x, \phi] = \phi_0 + \phi_1 a[\theta_{10} + \theta_{11}x] + \phi_2 a[\theta_{20} + \theta_{21}x] + \phi_3 a[\theta_{30} + \theta_{31}x] \quad (\text{P 3.1})$$

- Represents a **family of functions**; parameters determine the particular function
- Given parameters: inference = run the equation
- Given data: define a least-squares loss, change parameters to minimise it — exactly as before

ML $\mathbf{w}^\top \phi(\mathbf{x})$: linear in fixed features

NN Eq. (3.1): linear in *parameterised* features

Notation: two textbooks, same model

Bishop & Bishop (2024), §6.1

$$a_j = \sum_i w_{ji}^{(1)} x_i + w_{j0}^{(1)}$$

$$z_j = h(a_j)$$

$$y_k = \sum_j w_{kj}^{(2)} z_j + w_{k0}^{(2)}$$

Prince (2023), §3.1

$$h_d = a[\theta_{d0} + \theta_{d1}x]$$

$$y = \phi_0 + \sum_d \phi_d h_d$$

All parameters collectively: ϕ

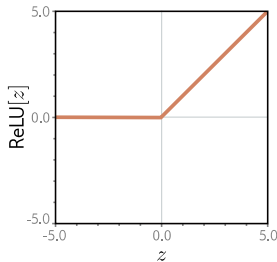
Eqs. 3.3–3.4

- Same model: $z_j = h(a_j) \equiv h_d = a[\cdot]$; Bishop's $\mathbf{w}^{(2)}$ $\equiv$ Prince's ϕ_d
- Careful: Prince's ϕ are **parameters**; Bishop's $\phi(\mathbf{x})$ are **fixed basis functions**
- Equations follow Bishop where possible; Prince's notation accompanies his figures

The activation function: ReLU

$$a[z] = \text{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}$$

- The **rectified linear unit**: clips negative values to zero
- Without it, Eq. (3.1) collapses to a single linear function of x



Prince (2023), Fig. 3.1.

ML fixed nonlinearity: x^j , Gaussian, sigmoid

NN nonlinearity applied to a *parameterised* line

Hidden units: break the model into two parts

Stage 1 — three linear functions, passed through the activation:

$$h_1 = a[\theta_{10} + \theta_{11}x], \quad h_2 = a[\theta_{20} + \theta_{21}x], \quad h_3 = a[\theta_{30} + \theta_{31}x] \quad (\text{P 3.3})$$

Stage 2 — weight and sum:

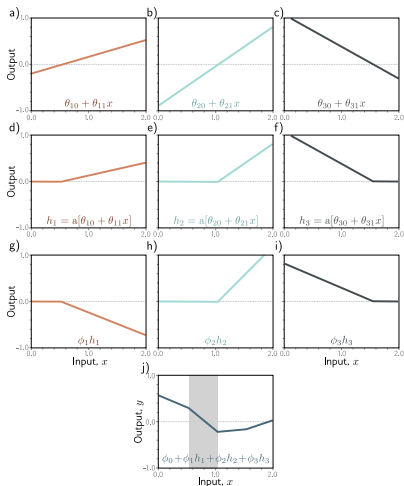
$$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3 \quad (\text{P 3.4})$$

- h_1, h_2, h_3 are the **hidden units** (Bishop: $z_j = h(a_j)$)
- Clipped $\rightarrow$ **inactive**; passing its input $\rightarrow$ **active**

ML regress y on fixed $\phi(x)$

NN Stage 2 is linear regression — on learned features h

Computing the function, step by step



a-c) compute three **linear** functions $\theta_{\bullet 0} + \theta_{\bullet 1}x$

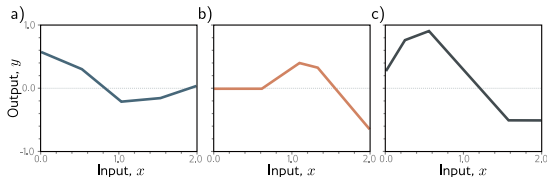
d-f) pass each through the **ReLU** $\rightarrow$ hidden units h_d

g-i) **weight** by ϕ_1, ϕ_2, ϕ_3

j) **sum** plus offset ϕ_0

Prince (2023), Fig. 3.3; sequence as in his Ch. 3 lecture.

The function family: piecewise linear



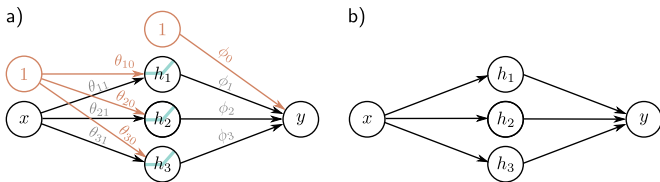
Prince (2023), Fig. 3.2.

- Continuous piecewise-linear functions
- One “joint” per ReLU: three units $\rightarrow$ up to four regions
- Each region $\leftrightarrow$ an activation pattern: which units are active

ML family: span of chosen basis functions

NN family: piecewise-linear, joints placed by training

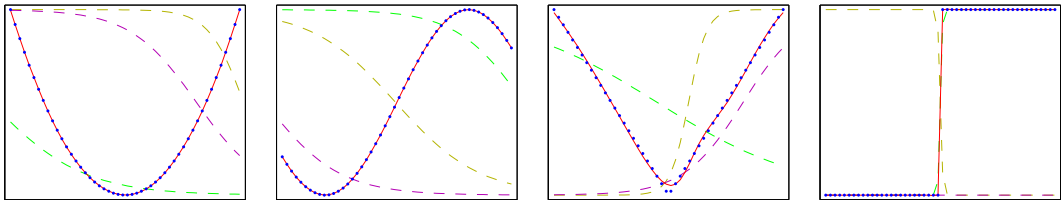
Depicting the network



Prince (2023), Fig. 3.4.

- Each parameter multiplies its source and adds to its target
- (b) is the standard shorthand: intercepts (offsets) usually not drawn

Bishop's picture: learned basis functions



Bishop & Bishop (2024), Fig. 6.10: dashed curves = hidden units z_j ($\tanh$); red curve = network output.

- Same architecture, $\tanh$ activation: hidden units act as **basis functions that adapt** to each dataset
- Four different target functions — the same network learns four different bases

Generalising the Network

Capacity: D hidden units

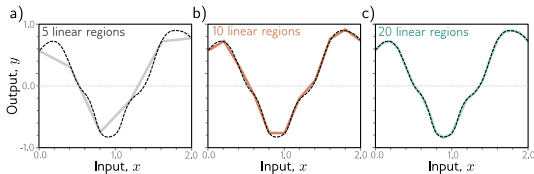
$$h_d = a[\theta_{d0} + \theta_{d1}x], \quad y = \phi_0 + \sum_{d=1}^D \phi_d h_d \quad (\text{P 3.5--3.6})$$

- The number of hidden units D is a measure of **network capacity**
- D ReLU units: D joints $\rightarrow$ at most **$D+1$ linear regions**

ML capacity dial: polynomial degree /
#basis functions M

NN capacity dial: hidden units D

Universal approximation

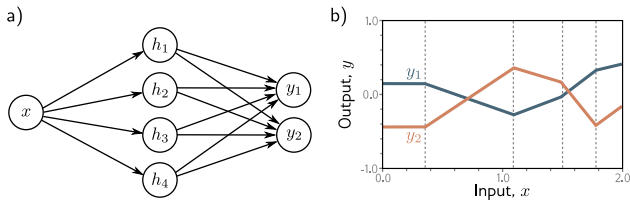


Prince (2023), Fig. 3.5.

With enough hidden units, a shallow network can describe **any continuous function** to arbitrary precision — the **universal approximation theorem** (Prince, 2023, §3.2).

- One more region per unit; more regions $\rightarrow$ closer fit
- Existence — not a recipe, not a size bound (Cybenko 1989; Hornik 1991)

More than one output



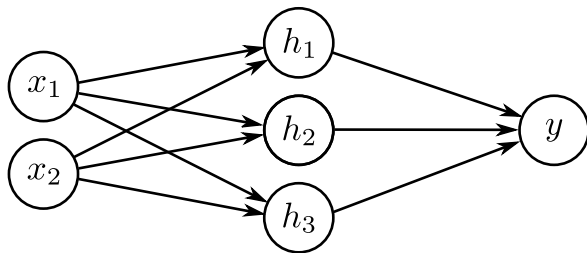
Prince (2023), Fig. 3.6.

1 input, 4 hidden units, 2 outputs:

$$y_k = \phi_{k0} + \sum_d \phi_{kd} h_d$$

- One extra linear read-out per output — hidden units are **shared**
- Hence both outputs have joints at the **same places**

More than one input



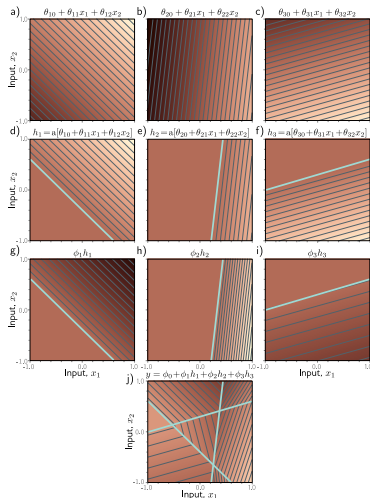
Prince (2023), Fig. 3.7.

2 inputs, 3 hidden units, 1 output:

$$h_d = a[\theta_{d0} + \theta_{d1}x_1 + \theta_{d2}x_2]$$

- One slope per input: each pre-activation is an **oriented plane** over the input space

Two inputs, computed step by step



a-c) three linear functions
(oriented planes) of x_1, x_2

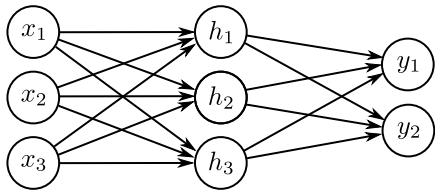
d-f) clipped by ReLU $\rightarrow$ hidden units

g-i) weighted by ϕ_d

j) summed + offset

The linear regions are **convex polygons**, one per activation pattern. Prince (2023), Fig. 3.8.

The general case



Prince (2023), Fig. 3.12.

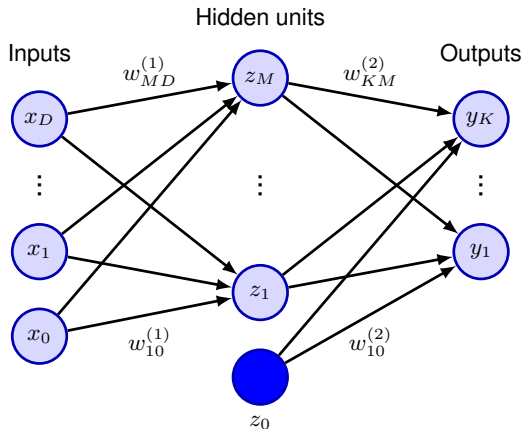
D_i inputs, D hidden units, D_o outputs (Bishop notation, Eqs. 6.1–6.3):

$$z_j = h\left(\sum_i w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right)$$

$$y_k = \sum_j w_{kj}^{(2)} z_j + w_{k0}^{(2)}$$

- e.g. three inputs, three hidden units, two outputs $\rightarrow$ 20 parameters

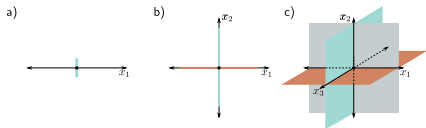
The same picture in Bishop's notation



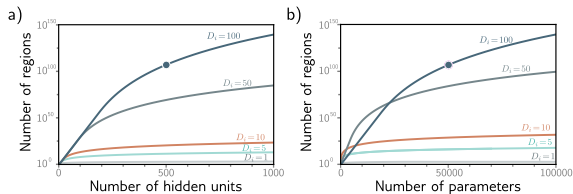
- Weights carry **layer superscripts**: $w_{ji}^{(1)}$ into the hidden layer, $w_{kj}^{(2)}$ out of it
- Biases drawn as links from clamped units $x_0 = 1, z_0 = 1$
- D inputs, M hidden units, K outputs — exactly Eqs. (6.1)–(6.3)

Bishop & Bishop (2024), Fig. 6.9.

How many regions?



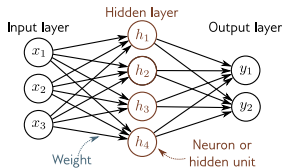
Prince (2023), Fig. 3.10: each hidden unit is a hyperplane cutting input space.



Prince (2023), Fig. 3.9.

- Zaslavsky (1975): D hyperplanes in D_i -dimensional input space create at most $\sum_{j=0}^{D_i} \binom{D}{j}$ regions – **binomial coefficients**
- Highlighted point: $D=500$ units (51,001 parameters) $\rightarrow$ more than 10^{107} regions

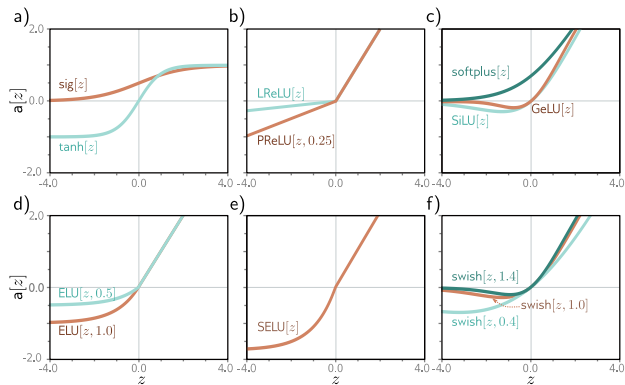
Terminology



Prince (2023), Fig. 3.11.

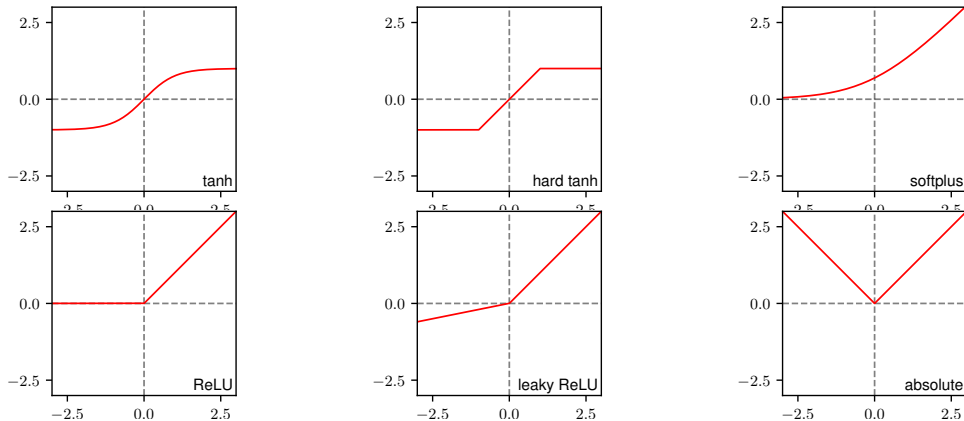
- input layer / hidden layer / output layer
- weights (slopes) and biases (offsets)
- pre-activations a_j , activations z_j
- fully connected, feed-forward; one hidden layer = shallow
- with ≥ 1 hidden layer, also called a multi-layer perceptron

Other activation functions



Prince (2023), Fig. 3.13: sigmoid, tanh, leaky ReLU, GeLU, SiLU, ELU, and others. ReLU is the modern default; tanh/sigmoid were historically standard.

Bishop's gallery of activations



Bishop & Bishop (2024), Fig. 6.12: tanh, hard tanh, softplus, ReLU, leaky ReLU, absolute.

Training

Same loss, harder landscape

Sum-of-squares error, now with a network inside (Bishop & Bishop, 2024, §6.1):

$$E(\mathbf{w}) = \frac{1}{2} \sum_n \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{t}_n\|^2$$

- Parameters sit inside $h(\cdot)$: $\nabla E = \mathbf{0}$ has **no closed form**
- The loss surface is **non-convex** — unlike every model so far
- Training: gradient descent, as on the Fig. 2.4 surface — Classes 11–13

ML normal equations: jump to the minimum, convex

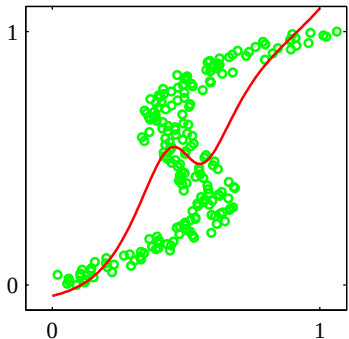
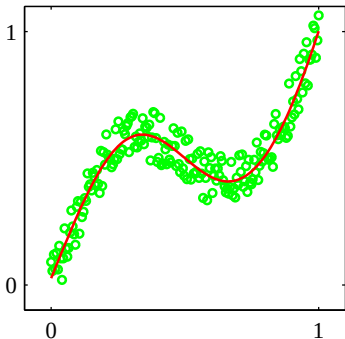
NN gradient descent: walk downhill, non-convex

Weight-space symmetries

The non-convexity is **structural** (Bishop & Bishop, 2024, §6.1.1):

- **Sign-flip**: flipping the sign of all weights into and out of a hidden unit gives the same function
- **Permutation**: relabelling hidden units gives the same function — $M!$ equivalent weight vectors
- Every minimum appears at least $M! \cdot 2^M$ times in weight space — the same function, many addresses

It works: shallow networks fit real data



Bishop & Bishop (2024), Fig. 6.17: two-layer network fits (red) to noisy data (green). Trained by gradient descent on the sum-of-squares error; the learned hidden units place their joints where the data demands.

The dictionary

	Linear/Basis-Function Model	Shallow Neural Network
Function class	lines, polynomials, RBFs	piecewise-linear surfaces
Features	$\phi_j(\mathbf{x})$, fixed	$z_j = h(a_j)$, learned
Prediction	$\mathbf{w}^T \phi(\mathbf{x})$	$\sum_j w_{kj}^{(2)} z_j + w_{k0}^{(2)}$
Loss (regression)	sum of squares	sum of squares
Optimisation	normal equations, convex	gradient descent, non-convex
Capacity	#basis functions M	hidden units D , regions $\leq \sum_j \binom{D}{j}$

A shallow network is regression whose basis functions carry parameters and are learned from data.

Takeaways

1. A shallow network puts parameters inside the basis functions; the output layer is a **linear regression on learned features**
2. With ReLU the family is **piecewise linear**: one joint per unit, regions counted by binomial coefficients
3. With enough units it approximates **any continuous function** — existence, not a recipe
4. The price: **no closed form**, a non-convex loss, and gradient-based training

Class 6: the same architecture for **classification** — sigmoid/softmax outputs and cross-entropy loss.

References

- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. §4.1, §6.1. Figures 4.1, 4.2, 6.10, 6.17 from the authors' published figure set, with attribution.
- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Ch. 2–3. Figures 2.2, 2.4, 3.1–3.13 and teaching sequence from the author's repository (<https://udlbook.github.io/udlbook/>), CC BY-NC-ND, with attribution.
- G. Cybenko (1989); K. Hornik (1991); T. Zaslavsky (1975).

Questions?